

Freinage d'urgence autonome (AEB) par fusion LiDAR–caméra

Conception, calibration expérimentale, validation sur véhicule
et portage de la perception sur ordinateur embarqué

Douae Sebti — Projet ProtoVA 2 — séances des 24 et 28 juillet 2026

1. Objectif et point de départ

L'objectif de la séance était de doter le véhicule d'un **freinage d'urgence autonome (AEB)** fiable : le véhicule doit s'arrêter seul devant tout obstacle, même si le conducteur maintient l'accélérateur. La solution précédente d'évitement (*follow the gap*, purement réactive sur le LiDAR) s'était montrée insuffisante : détection incomplète des obstacles et collisions avec les murs. J'ai donc repris le problème à la base, en m'appuyant sur les capteurs embarqués (caméra de profondeur et LiDAR) et en validant chaque choix par la mesure.

2. Démarche : de la caméra seule à la fusion de capteurs

2.1 Détection générique par la profondeur

La détection YOLO seule ne convient pas pour un AEB : elle ne reconnaît que les classes apprises (piéton, chaise...) et **ignore les murs**. J'ai donc ajouté au nœud de perception une mesure directe sur l'**image de profondeur** : la distance au plus proche obstacle dans un couloir frontal est extraite de la carte de profondeur brute (percentile bas, zone centrale), indépendamment de toute classification. Tout obstacle, connu ou non, est ainsi détecté.

2.2 Limite de la caméra et fusion avec le LiDAR

Les essais ont montré une limite physique de la caméra temps-de-vol : les surfaces absorbant l'infrarouge (caoutchouc, matériaux sombres) renvoient une profondeur invalide et deviennent invisibles. J'ai donc conçu un nœud d'AEB par **fusion de capteurs**, exécuté localement sur la Jetson (réaction ~30 Hz, sans dépendre du PC ni du réseau) : le LiDAR (fiable sur toutes les surfaces, balayage 360°) fournit la distance au plus proche obstacle dans les secteurs avant *et* arrière, la caméra complète sur le couloir frontal, et la distance de danger retenue est le **minimum des deux capteurs** pour chaque direction.

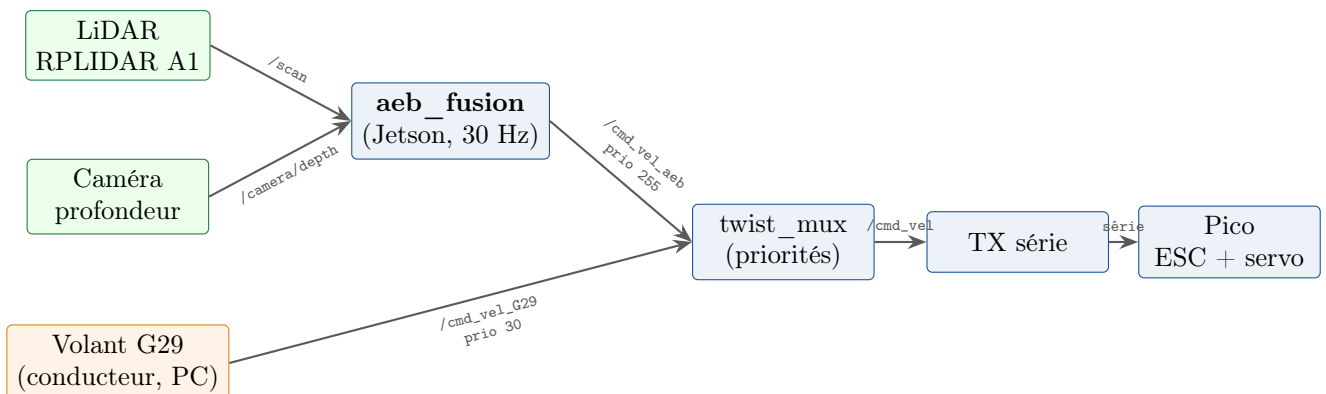


FIGURE 1 — Architecture de l'AEB : la fusion capteurs s'insère dans la chaîne de commande existante via le multiplexeur, avec la priorité maximale — l'arrêt d'urgence l'emporte toujours sur le conducteur.

Un point matériel important a été identifié au passage : le LiDAR étant monté « moteur à l'arrière », l'avant réel du véhicule correspond à **180°** du repère du scan. Je l'ai vérifié sur données

réelles (obstacle à 0,67 m derrière contre 3,2 m d'espace libre devant) et corrigé dans tous les nœuds concernés (figure 2).

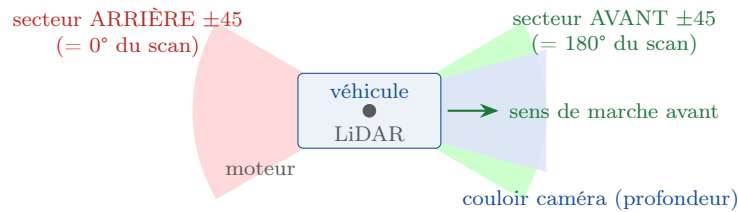


FIGURE 2 – Vue de dessus des zones surveillées. Le LiDAR balaie 360° : j'en extrais un secteur avant et un secteur arrière ($\pm 45^\circ$) ; la caméra couvre un couloir frontal. Le montage « moteur à l'arrière » décale l'avant réel à 180° du repère du scan.

3. Calibration du freinage au banc d'essai

Premier constat en essai : couper les gaz ne freine pas — le variateur (ESC) passe en **roue libre** et le véhicule continue sur son élan. J'ai donc mis en place un **banc de mesure** : roues levées, un script publie une consigne d'avance pendant 2 s puis déclenche la stratégie de freinage à évaluer, et le **temps d'arrêt est chronométré sur l'encodeur de roue** (mesure objective, indépendante du ressenti). Résultats :

Stratégie de freinage	Temps d'arrêt	Gain
Roue libre (gaz coupés)	1,94 s	—
Contre-impulsion -0,3 / 0,25 s	1,16 s	-40 %
Contre-impulsion -0,5 / 0,5 s	0,61 s	-69 %
Contre-impulsion -0,8 / 0,5 s	0,57 s	saturation ESC

Le réglage -0,5 pendant 0,5 s est retenu (au-delà, l'ESC sature). L'impulsion est **interrompue dès l'arrêt réel** mesuré par l'encodeur, et n'est appliquée qu'en marche avant : en freinage de marche arrière, l'arrêt est direct, sans aucun mouvement inverse parasite.

4. Comportement final de l'AEB

- **Bidirectionnel** : marche avant freinée par les obstacles avant, marche arrière par les obstacles arrière ; les deux distances sont affichées en continu.
- **Prioritaire** : priorité maximale dans le multiplexeur — le véhicule **reste arrêté même accélérateur enfoncé**.
- **Échappatoire** : la direction opposée au danger reste toujours disponible.
- **Distance de freinage dynamique** : les seuils croissent avec la vitesse réelle mesurée (figure 3) — plus le véhicule roule vite, plus il freine tôt.
- **Réactivité** : délai de reprise en main après un arrêt ramené de 1 s à 0,3 s (temporisation du multiplexeur).

5. Validation par « boîte noire » embarquée

5.1 Principe

En essais de conduite, le ressenti du pilote ne suffit pas : « la voiture ne s'arrête pas » peut aussi bien venir d'un capteur, de la décision de l'AEB, du multiplexeur, de la liaison série ou de la

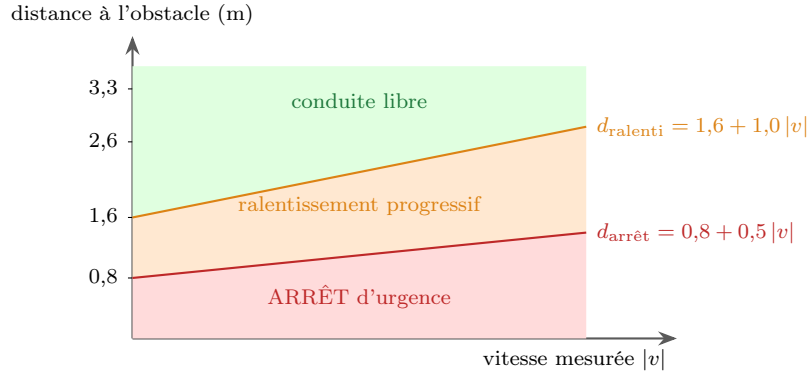


FIGURE 3 – Seuils de freinage dynamiques : les frontières d’arrêt et de ralentissement montent avec la vitesse réelle du véhicule (mesurée par l’encodeur), comme sur un AEB automobile.

mécanique. Pour trancher, j’ai appliqué la méthode des **enregistreurs de données automobiles** (EDR / data logger) : capturer *simultanément*, pendant la conduite, toutes les grandeurs de la chaîne, puis relire la chronologie à froid.

5.2 Mise en œuvre

L’outil est un **nœud ROS 2 en Python** (bibliothèque `rclpy`, sans logiciel externe) que j’exécute sur la Jetson pendant l’essai. Il s’abonne en parallèle à cinq sources et échantillonne toutes les 0,5 s pendant 90 s :

Colonne	Sujet ROS 2	Grandeur	Maillon observé
AVANT / ARR	<code>/scan</code>	distances min. des 2 secteurs	perception LiDAR
pédale	<code>/cmd_vel_G29</code>	consigne du conducteur	volant G29
AEBpub	<code>/cmd_vel_aeb</code>	sortie de l’AEB	décision
SORTIE	<code>/cmd_vel</code>	sortie du multiplexeur	arbitrage
roues	<code>/velocity</code>	vitesse encodeur	réalité mécanique

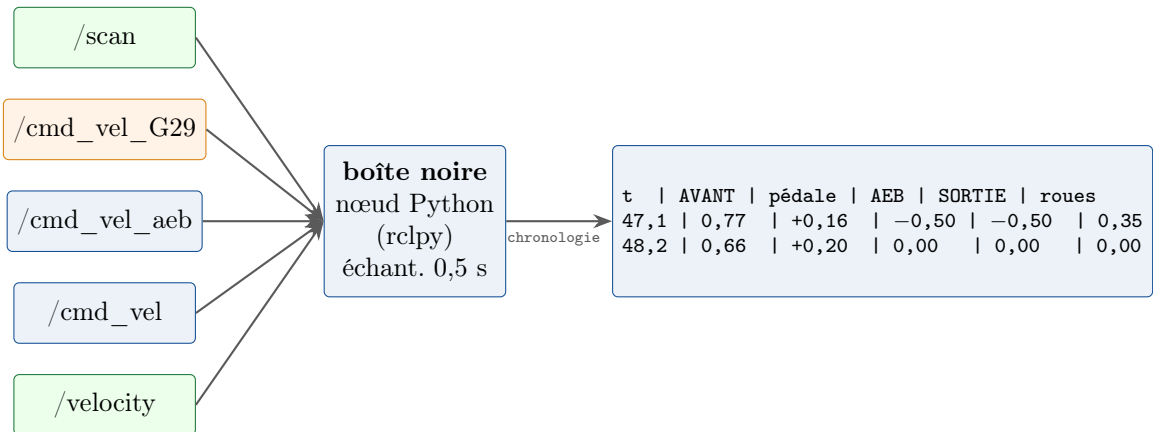


FIGURE 4 – La « boîte noire » : un nœud d’observation passif s’abonne à toute la chaîne en parallèle et produit une chronologie synchronisée, relue après l’essai.

En pratique : je lance l’enregistreur (en tâche de fond, via SSH), j’effectue la manœuvre à valider (accélérer vers l’obstacle en gardant la pédale, tenter de redémarrer...), puis j’analyse le tableau produit.

5.3 Ce que l'enregistrement a démontré

t=47,1 s avant 0,77 m pédale +0,16 AEB -0,50 (impulsion de frein) roues 0,35
 t=48,2 s avant 0,66 m pédale +0,20 (enfoncée) AEB 0,00 roues 0,00 - arrêté

- l'arrêt complet **accélérateur enfoncé** est confirmé (pédale +0,20, roues à 0) — le ressenti contraire venait d'essais antérieurs ;
- le « non-redémarrage » ressenti est objectivé : **1 s de latence** entre le relâchement de l'AEB et le retour de la main au conducteur (temporisation du multiplexeur), corrigée ensuite (0,3 s) ;
- le freinage arrière s'arrête net, sans avancée parasite.

Cette méthode ressenti → mesure → correction s'est révélée décisive à chaque étape de la séance ; je la réutiliserai pour les prochains réglages.

6. Portage de la perception sur le calculateur embarqué

6.1 Constat

La détection YOLO s'exécutait initialement sur le PC : les images de la caméra traversaient le WiFi, l'inférence tournait sur le processeur, et la visualisation accusait un retard de l'ordre de 0,5 à 0,8 s. Ce retard n'affectait pas la sécurité (le freinage est calculé localement sur la Jetson, sans passer par le réseau), mais il pénalisait la supervision et interdisait toute décision rapide fondée sur la classification des objets.

6.2 Démarche

J'ai d'abord vérifié que le PC ne disposait pas d'accélérateur exploitable (carte graphique AMD, CUDA indisponible pour PyTorch). J'ai donc porté la détection sur le **GPU de la Jetson Orin** : installation de la pile CUDA (JetPack 6.2.1 : CUDA 12.6, cuDNN, TensorRT), puis de PyTorch dans sa version compilée pour Jetson et de la bibliothèque de détection. J'ai ensuite mesuré le gain avec un banc d'inférence dédié :

Exécution	Temps d'inférence	Cadence	Gain
Processeur (Jetson)	344,1 ms / image	2,9 im/s	—
GPU CUDA (Jetson Orin)	30,3 ms / image	33 im/s	× 11

6.3 Architecture retenue

La détection s'exécute désormais **à bord**, à 10 Hz (contre 5 Hz auparavant), et les images ne transitent plus par le réseau pour être analysées. Pour conserver la supervision depuis le poste de pilotage sans réintroduire de latence, le nœud publie une **image déjà annotée, compressée en JPEG** : seule cette image légère traverse le WiFi. Le véhicule gagne ainsi en autonomie — il peut classifier un piéton et déclencher une alerte V2V sans dépendre du PC.

6.4 Difficultés rencontrées

L'installation a demandé plusieurs corrections : le dépôt de paquets NVIDIA pour Jetson avait changé d'adresse ; la résolution automatique des dépendances installait une version générique de PyTorch *dépourvue* du support CUDA pour Jetson (il a fallu installer explicitement les archives officielles) ; une bibliothèque d'algèbre creuse manquait au chargement ; enfin, la version de la bibliothèque de calcul numérique a dû être choisie avec soin pour rester compatible avec ROS 2.

7. Bilan et suites

Bilan. Le véhicule dispose désormais d'un freinage d'urgence autonome bidirectionnel, fondé sur la fusion LiDAR-caméra, calibré expérimentalement (temps d'arrêt réduit de 1,94 s à 0,61 s) et validé en conduite réelle par enregistrement embarqué. La chaîne de direction a par ailleurs été vérifiée (balayage commandé du servomoteur), et la perception visuelle a été portée sur le GPU du calculateur embarqué (inférence 11 fois plus rapide). L'ensemble du code, des scripts de banc et des mesures est archivé sur le dépôt Git du projet.

Suites envisagées.

- affiner les coefficients de seuil dynamique sur davantage d'essais au sol ;
- exploiter la classification embarquée pour enrichir l'alerte V2V (type d'obstacle) ;
- intégrer l'AEB à la démonstration V2V complète (freinage → message DENM vers le véhicule voisin).